



Nuclei™ Eval SoC Introduction

Nuclei

All rights reserved by Nucleisys, Inc.

Contents

1	Copyright Notice	1
2	Contact Information	2
3	Revision History	5
4	Eval_SoC Background	6
5	Eval_SoC Diagram	8
6	Memory Resources of Eval_SoC	9
6.1	On-Chip SRAM Resource	9
6.2	External Flash Support	10
6.3	DDR	10
7	Address Allocation of Eval_SoC	11
8	Reset PC Address	12
9	Ports of Eval_SoC	13
10	Interrupts of Eval_SoC	15
11	Peripherals of Eval_SoC	16
11.1	GPIO	16
11.2	QSPI	17
11.3	UART	17
12	Appendix	18

Copyright Notice

Copyright© 2018-2021 Nuclei System Technology. All rights reserved.

Nuclei are trademarks owned by Nuclei System Technology. All other trademarks used herein are the property of their respective owners.

The product described herein is subject to continuous development and improvement; information herein is given by Nuclei in good faith but without warranties.

This document is intended only to assist the reader in the use of the product. Nuclei System Technology shall not be liable for any loss or damage arising from the use of any information in this document, or any incorrect use of the product.

Contact Information

Should you have any problems with the information contained herein or any suggestions, please contact Nuclei System Technology by email support@nucleisys.com, or visit “Nuclei User Center” website <http://user.nucleisys.com> for supports or online discussion.

List of Figures

4.1	Example case 1 of Nuclei Subsystem IP Solution	7
4.2	Example case 2 of Nuclei Subsystem IP Solution	7
5.1	Eval_SoC Diagram	8

List of Tables

7.1	Address Allocation of Eval_SoC	11
9.1	Ports of SoC	13
10.1	Allocation of External Interrupts in Eval_SoC	15
11.1	GPIO Pin-Mux	16

Revision History

Rev.	Revision Date	Revised Section	Revised Content
1.0.0	2020/1/20	N/A	1.First version as the full English
1.1.0	2021/12/14	All Sections	1.Refine the content as the Eval_SoC adopts Nuclei SoC Subsystem

Eval_SoC Background

To easy user to evaluate Nuclei Processor Core, the prototype SoC (called Eval_SoC) is provided for evaluation purpose. This Eval_SoC includes:

- Processor Core, it can be Nuclei N class, NX class or UX class Processor Core.
- On-Chip SRAMs for instruction and data.
- The SoC buses.
- The basic peripherals, such as UART, GPIO, SPI.

With this prototype SoC, user can run simulations, map it into the FPGA board, and run with real embedded application examples. Currently in Nuclei SDK and Nuclei Studio, the target name of Eval_SoC is “demosoc”.

Please note that all modules including the peripherals in the Eval_SoC are for evaluation purpose and without Quality Guarantee/Supports, hence, it is not recommended to be used in any productions.

If user really needs the SoC solution and peripherals IP with Quality Guarantee/Supports, Nuclei provides SoC Subsystem solution which can help user to quickly make the SoC product. Here are two example cases of Subsystem IP solutions provided by Nuclei

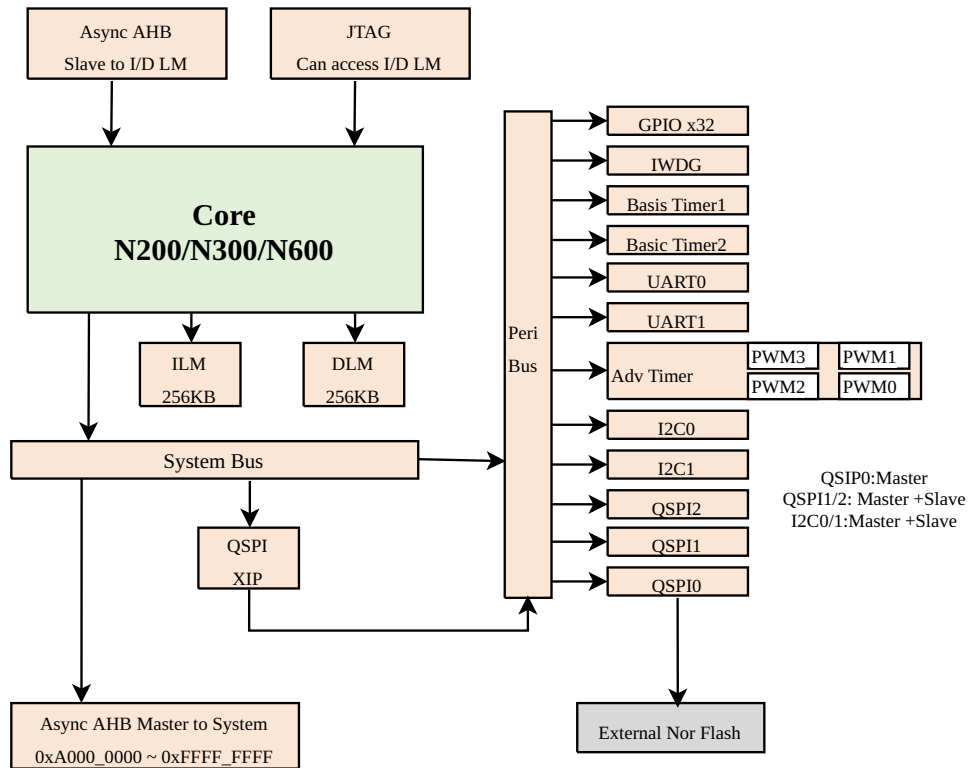


Fig. 4.1: Example case 1 of Nuclei Subsystem IP Solution

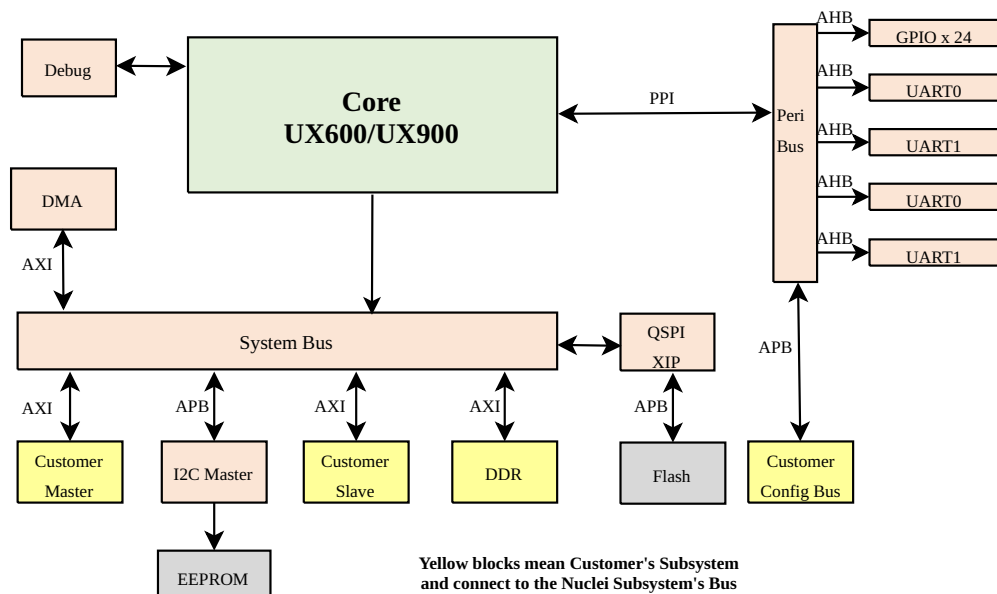


Fig. 4.2: Example case 2 of Nuclei Subsystem IP Solution

Nuclei can provide a concrete SoC Subsystem based on user's requirements, also we can provide testbench, FPGA Bitstream, scripts for simulation/synthesis, SDK with peripheral drivers and test cases, documents in the deliver package. For more details, please visit <https://www.nucleisys.com/subsystem.php> or contact Nuclei Support.

Eval_SoC Diagram

The Eval_SoC diagram is as depicted in [Eval_SoC Diagram](#) (page 8). There are Processor Core, Buses (System Bus and Peripheral Bus) and Peripherals in the SoC.

Note:

- Only UX Series Cores has the DDR, and the DDR uses the FPGA's.
- If there is no ILM and DLM interface configured for core, then ILM and DLM will be only connected to System Bus.
- If there is no FIO (Fast-IO) configured for core, then the GPIO will be connected to Peripheral Bus.
- If there is no PPI configured for core, then the Peripheral Bus will be connected to System Bus.

For more information of the peripherals, please refer to [Peripherals of Eval_SoC](#) (page 16).

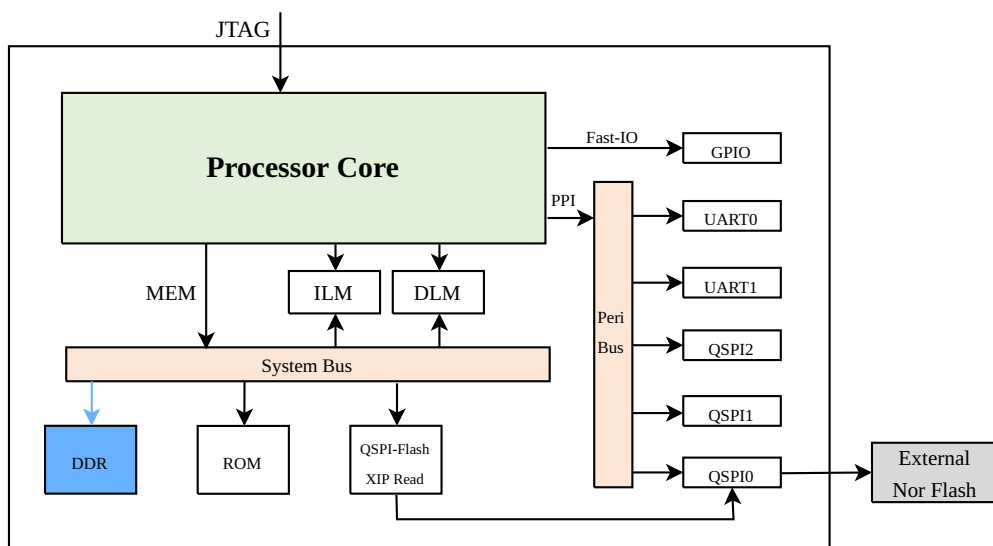


Fig. 5.1: Eval_SoC Diagram

Memory Resources of Eval_SoC

The Eval_SoC contained memory resources including on-chip SRAM resource, External Flash support and DDR (DDR only exists in FPGA Board of UX Core's Eval_SoC).

6.1 On-Chip SRAM Resource

The Eval_SoC contained on-chip SRAM and ROM, detailed as below.

- ILM with features:
 - Size configurable.
 - Mainly used to save the instructions, but also can be used to save the data.
 - Base address is allocated as shown in *Address Allocation of Eval_SoC* (page 11).
 - The ILM is connected to system bus, so the Core can access it with system bus interface.
- DLM with features:
 - Size configurable.
 - Mainly used to save the data.
 - Base address is allocated as shown in *Address Allocation of Eval_SoC* (page 11).
 - The DLM is connected to system bus, so the Core can access it with system bus interface.
- ROM with features:
 - Size is 4KB.
 - The ROM is connected to system bus, so the Core can access it with system bus interface.
 - There is just 1~2 instructions in ROM, after executed the instructions, Core will to jump to ILM (address at 0x8000_0000). That is to say, in SIMULATION environment, the Core will just jump to ILM after reset and start execution from ILM.

6.2 External Flash Support

The Eval_SoC supports external SPI Nor Flash with QSPI0 interface, detailed as below.

- The QSPI0 support the eXecute-In-Place (XIP) mode, and it is the default mode after reset.
- With XIP mode, external Flash can be treated as a read-only memory directly accessible (address space is 0x2000_0000 ~ 0x3FFF_FFFF in this SoC).
- Hence, the instructions program can be saved at external Flash (will not be lost even after power down), and then after power-on reset, the Processor Core can fetch instruction directly from the external Flash with XIP mode through this QSPI master interface, and start to execute the programs.
- Please refer to [QSPI](#) (page 17) for more info of QSPI XIP mode.

6.3 DDR

The Eval_SoC of UX Core has DDR , detailed as below:

- Currently UX Core's Eval_SoC uses FPGA Board's DDR, the FPGA Board of UX Core's Eval_SoC can easy customer to boot Linux.
- The DDR only exists in FPGA Board (not in Simulation Environment), the base address is 0x6000_0000, the size depends on the specific FPGA Board.

Address Allocation of Eval_SoC

The address allocation of the Eval_SoC is as shown in [Address Allocation of Eval_SoC](#) (page 11).

Table 7.1: Address Allocation of Eval_SoC

	Component	Address Spaces	Description
Core Private Peripherals	TIMER	0x0200_0000 ~ 0x0200_0FFF	TIMER Unit address space.
	ECLIC	0x0C00_0000 ~ 0x0C00_FFFF	ECLIC Unit address space.
	DEBUG	0x0000_0000 ~ 0x0000_0FFF	DEBUG Unit address space.
Memory Resource	ILM	0x8000_0000 ~	ILM address space.
	DLM	0x9000_0000 ~	DLM address space.
	ROM	0x0000_1000 ~ 0x0000_1FFF	Internal ROM.
	Off-Chip QSPI0 Flash Read	0x2000_0000 ~ 0x3FFF_FFFF	QSPI0 with XiP mode read-only address space.
	DDR	0x6000_0000 ~	DDR address space. (Only for FPGA Board of UX Core's Eval_SoC)
Peripherals	GPIO	0x1001_2000 ~ 0x1001_2FFF	GPIO Unit address space.
	UART0	0x1001_3000 ~ 0x1001_3FFF	First UART address space.
	QSPI0	0x1001_4000 ~ 0x1001_4FFF	First QSPI address space.
	UART1	0x1002_3000 ~ 0x1002_3FFF	Second UART address space.
	QSPI1	0x1002_4000 ~ 0x1002_4FFF	Second QSPI address space.
	QSPI2	0x1003_4000 ~ 0x1003_4FFF	Third QSPI address space.
Default slave	The other space is write-ignored and read-as zero.		

Reset PC Address

After reset, the reset PC address of the Processor Core could be from external Flash (XIP mode), or from the internal ROM.

- In FPGA board, the reset PC address is from external Flash (XIP)
 - In the SoC, “QSPI Flash XIP Read” bus slave port have been allocated with address space of 0x2000_0000 ~ 0x3FFF_FFFF at system bus.
 - In the FPGA version of SoC RTL code, there is a macro check “`ifdef FPGA_SOURCE`”, and in this case the input pin “reset_vector” of Processor Core is tied as value “0x2000_0000”, then after reset, the Core will start execution from “QSPI Flash XIP Read”, i.e., the external Flash.
 - Please refer to [Eval_SoC Diagram](#) (page 8) for more information of the Eval_SoC diagram, and refer to [QSPI](#) (page 17) for more info of QSPI XIP mode.
- In SIMULATION Environment, the reset PC address is from internal ROM.
 - In the SoC, “internal ROM” bus slave port have been allocated with address space of 0x0000_1000 ~ 0x0000_1FFF at system bus.
 - In the SIMULATION version of SoC RTL code, there is a macro check “`ifndef FPGA_SOURCE`”, and in this case the input pin “reset_vector” of Processor Core is tied as value “0x0000_1000”, then after reset, the Core will start execution from “Internal ROM”. There is just 1~2 instructions in ROM, after executed the instructions, Core will to jump to ILM (address at 0x8000_0000). That is to say, in SIMULATION environment, the Core will just jump to ILM after reset and start execution from ILM.
 - Please refer to [Eval_SoC Diagram](#) (page 8) for more information of the Eval_SoC diagram.

Ports of Eval_SoC

The top level ports of the SoC are as shown in *GPIO Pin-Mux* (page 16).

Table 9.1: Ports of SoC

Direction	Name	Description
Input	JTAG TCK	JTAG TCK Signal
Output	JTAG TDO	JTAG TDO Signal
Output	JTAG_DRV_TDO	JTAG TDO Output Enable
Input	JTAG TMS	JTAG TMS Signal
Input	JTAG TDI	JTAG TDI Signal
Bidir	QSPI DQ 3	Quad SPI Data3
Bidir	QSPI DQ 2	Quad SPI Data2
Bidir	QSPI DQ 1	Quad SPI Data1
Bidir	QSPI DQ 0	Quad SPI Data0
Output	QSPI CS	Quad SPI Chip Select
Output	QSPI SCK	Quad SPI Clock
Bidir	GPIO_0	32 General Purpose I/O
Bidir	GPIO_1	
Bidir	GPIO_2	
Bidir	GPIO_3	
Bidir	GPIO_4	
Bidir	GPIO_5	
Bidir	GPIO_6	
Bidir	GPIO_7	
Bidir	GPIO_8	
Bidir	GPIO_9	
Bidir	GPIO_10	
Bidir	GPIO_11	
Bidir	GPIO_12	
Bidir	GPIO_13	
Bidir	GPIO_14	
Bidir	GPIO_15	
Bidir	GPIO_16	
Bidir	GPIO_17	
Bidir	GPIO_18	

continues on next page

Table 9.1 – continued from previous page

Direction	Name	Description
Bidir	GPIO_19	
Bidir	GPIO_20	
Bidir	GPIO_21	
Bidir	GPIO_22	
Bidir	GPIO_23	
Bidir	GPIO_24	
Bidir	GPIO_25	
Bidir	GPIO_26	
Bidir	GPIO_27	
Bidir	GPIO_28	
Bidir	GPIO_29	
Bidir	GPIO_30	
Bidir	GPIO_31	

Interrupts of Eval_SoC

The interrupts of Eval_SoC is managed by the interrupt controller of the core, the interrupts are directly connected to the external interrupt interface of the processor Core.

In this Eval_SoC, only the GPIO connected to the Core as 32 external interrupt sources, as shown in *GPIO Pin-Mux* (page 16).

Table 10.1: Allocation of External Interrupts in Eval_SoC

External Interrupts to the Core	Source
0	gpio_0
...	...
31	gpio_31

Peripherals of Eval_SoC

The Chapter will shortly introduce the peripherals used in the Eval_SoC.

11.1 GPIO

There is a GPIO (General Purpose I/O) in the SoC. GPIO provide 32 general purpose I/O ports, the key points of which are:

- Each I/O can be programmed as input or output. If as output, the output value can also be programmed.
- Each I/O can be as the interrupt source, connecting to interrupt controller of core.
- Each I/O can be programmed to IOF (Hardware I/O Functions) mode, in this mode, the I/O is worked as Pin-Mux to serve internal peripherals, e.g., SPI, UART, etc. Each pin can be reused by two different sources, called IoF0 and IoF1. The Pin-Mux table is as shown in *GPIO Pin-Mux* (page 16).

Table 11.1: GPIO Pin-Mux

GPIO Pin Number	IOF0	IOF1
4	QSPI1:SD1/MISO	
5	QSPI1:SCK	
6	QSPI1:SD2	
7	QSPI1:SD3	
8	QSPI1:SS1	
9	QSPI1:SS2	
10	QSPI1:SS3	
16	UART0:RX	
17	UART0:TX	
24	UART1:RX	
25	UART1:TX	
26	QSPI2:SS	
27	QSPI2:SD0/MOSI	
28	QSPI2:SD1/MISO	
29	QSPI2:SCK	
30	QSPI2:SD2	
31	QSPI2:SD3	

Note:

- **This GPIO in the Eval_SoC is for evaluation purpose and without Quality Guarantee/Supports, hence, it is not recommended to be used in any productions.**
- **If user really needs the SoC solution or GPIO with Quality Guarantee/Supports, Nuclei provides SoC Subsystem which can help user to quickly make the SoC product. Please visit <https://www.nucleisys.com/subsystem.php> or contact Nuclei Support.**

11.2 QSPI

There are 3 independent QSPI in the SoC, which all use GPIO Pin-Mux to communicate with outside.

- One QSPI for Flash:
 - There is one QSPI master in the SoC which has dedicated SoC ports to communicate with external FLASH (e.g. Nor Flash with SPI interface).
 - This QSPI support the eXecute-In-Place (XIP) mode, and it is the default mode after reset. With XIP mode, external Flash can be treated as a read-only memory directly accessible. Hence, the instructions program can be saved at external Flash (will not be lost even after power down), and then after power-on reset, the Processor Core can fetch instruction directly from the external Flash with XIP mode through this QSPI master interface, and start to execute the programs.
- Another two QSPI:
 - There are two QSPI master (without XIP mode supported), which all use GPIO Pin-Mux to communicate with outside. One QSPI master has 4 CS (Chip Select) signals, another has 1 CS signal.

Note:

- **This QSPI in the Eval_SoC is for evaluation purpose and without Quality Guarantee/Supports, hence, it is not recommended to be used in any productions.**
- **If user really needs the SoC solution or QSPI with Quality Guarantee/Supports, Nuclei provides SoC Subsystem which can help user to quickly make the SoC product. Please visit <https://www.nucleisys.com/subsystem.php> or contact Nuclei Support.**

11.3 UART

There are 2 independent UART (Universal Asynchronous Receiver-Transmitter) in the SoC, which all use GPIO Pin-Mux to communicate with outside.

Note:

- **This UART in the Eval_SoC is for evaluation purpose and without Quality Guarantee/Supports, hence, it is not recommended to be used in any productions.**
- **If user really needs the SoC solution or UART with Quality Guarantee/Supports, Nuclei provides SoC Subsystem which can help user to quickly make the SoC product. Please visit <https://www.nucleisys.com/subsystem.php> or contact Nuclei Support.**

- **Nuclei RISC-V IP Products:** <https://www.nucleisys.com/product.php>
- **Nuclei Spec Documentation:** <https://nucleisys.com/download.php#spec>
- **Nuclei RISC-V Tools and Documents:** <https://nucleisys.com/download.php>
- **Nuclei Prebuilt Toolchain and IDE:** <https://nucleisys.com/download.php#tools>
- **NMSIS:** <https://github.com/Nuclei-Software/NMSIS>
- **Nuclei SDK:** <https://github.com/Nuclei-Software/nuclei-sdk>
- **Nuclei Linux SDK:** <https://github.com/Nuclei-Software/nuclei-linux-sdk>
- **Nuclei Software Organization in Github:** <https://github.com/Nuclei-Software/>
- **RISC-V MCU Organization in Github:** <https://github.com/riscv-mcu/>
- **RISC-V MCU Community Website:** <https://www.rvmcu.com/>
- **Nuclei riscv-openocd:** <https://github.com/riscv-mcu/riscv-openocd>
- **Nuclei riscv-gnu-toolchain:** <https://github.com/riscv-mcu/riscv-gnu-toolchain>